

Micetro External Discovery

Configuration & Operations Guide

Script version 0.9.35

Document version 1.1

Contents

1. Overview

2. How It Works

3. Configuration — `setup.json`

4. Configuration — `profiles.json`

5. Installation

6. Appendix

1. Overview

externalDiscovery is a Python script that extends Micetro's built-in network discovery. It fills the gaps left by the native SNMP engine: vendor APIs, VRF-aware discovery, Layer-2 (BRIDGE-MIB) discovery, and cleanup of noisy /31–/32 loopback networks.

Why use it instead of built-in discovery

- SNMP doesn't work reliably against PaloAlto or FortiGate, but their native APIs do
- VRF support is needed (tested on Cisco IOS and Nexus)
- Loopback addresses configured inside a range create unwanted /31 or /32 networks
- A non-standard SNMP OID is needed to retrieve ARP or subnet data
- Specific subnets or IPs need to be filtered out of discovery
- Layer-2 port/VLAN mapping is needed via the SNMP BRIDGE-MIB
- NSX-T segment/subnet and VM IP data lives behind its own REST API, not SNMP

Tested platforms

PAN-OS · Cisco Nexus (NX-OS) · Cisco IOS · VyOS · Netgear · FortiOS · NSX-T

What's in the package

File	Purpose
externalDiscovery.py	Python source. Not recommended for direct use in customer environments.
externalDiscovery	Compiled x64 Linux binary (built with PyInstaller).
externalDiscovery.exe	Compiled x64 Windows binary (built with PyInstaller).
setup.json	Configuration for the Micetro environment itself. One per install.
profiles.json	Configuration for the routers/switches to be discovered.

2. How It Works

At a high level, each run of the script:

- Reads setup.json (Micetro connection details) and profiles.json (device list)
- Logs in to Micetro and, if needed, creates the custom properties it depends on
- Groups devices by profile, then processes each profile's discoveryType (SNMP, NX-OS, PAN-OS, FORTI-OS, or NSX-T) using one thread per device
- For each device: reads interfaces/subnets, reads ARP (or DHCP/MAC tables), and writes the results back to Micetro via its SOAP-style API
- Optionally checks for devices that haven't been seen recently and raises an alert (log + syslog)

Note: A PID file prevents overlapping runs. If a previous run is still "in progress" past maxPIDAge, the stale PID file is removed automatically.

3. Configuration — setup.json

setup.json holds everything specific to the Micetro install itself, one file per environment.

Minimal example

```
{
  "url": "https://micetro.example.com/mmws/api/command/",
  "validateCert": false,
  "skipCIDRs": ["31", "32"],
  "skipIPs": ["127.0.0.1", "0.0.0.1", "254.0.0.1"],
  "skipNetworks": ["127.0.0.1", "10.24.32.0"],
  "vrfOverlap": false,
  "logFileName": "snmp.log",
  "logFileSizeInBytes": 10485760,
  "logFileBackupCount": 5,
  "encryptAuth": true,
  "fernetUser": "Fernet",
  "logLevel": 10,
  "timeout": 5,
  "snmpRetries": 3
}
```

Core settings

Key	Description
url	Location of the Micetro SOAP API, e.g. https://micetro.example.com/mmws/api/command/
validateCert	true validates the HTTPS certificate (use with a real PKI cert); false skips validation (self-signed).
skipCIDRs	Subnet masks to exclude from network creation, e.g. ["31","32"] — stops loopback /31–/32s polluting the GUI. ARP entries in a skipped CIDR are rolled up into the parent network instead.
skipIPs / skipNetworks	Specific IPs or networks that should never be created in Micetro.
vrfOverlap	true if discovered VRFs reuse overlapping address space and need separate namespaces (not yet supported); false if all VRFs use unique addressing.
logFileName	Log file path, relative to the folder one level above /scripts.
logFileSizeInBytes / logFileBackupCount	Log rotation size and number of rotated copies to keep.
logLevel	Numeric Python logging level — see table below.
timeout	Seconds to wait for an unresponsive host before giving up.
snmpRetries	Number of retries before an SNMP query is treated as failed.
maxPIDAge	Minutes before a stale PID file is assumed to be from an interrupted run and deleted.

Log levels

Level	Value	Use it for
NOTSET	0	Defer to ancestor loggers; effectively logs everything if unresolved.

Level	Value	Use it for
DEBUG	10	Full detail — recommended while first configuring a profile.
INFO	20	Confirmation that things are working normally.
WARNING	30	Something unexpected, but the script is still functioning.
ERROR	40	A specific operation failed.
CRITICAL	50	Serious error — the script may be unable to continue.

Encryption of stored credentials

- `encryptAuth: true` — SNMPv3 and API credentials in `profiles.json` are encrypted at rest using a Fernet key (see cryptography.io/en/latest/fernet/). The key is generated on first run and stored in the description field of the `fernetUser` account in Micetro. Any clear-text credentials found are encrypted automatically and `profiles.json` is rewritten with the encrypted values (prefixed `EEEEEE#`).
- `fernetUser` — the Micetro account used to hold that key.
- `encryptAuth: false` — credentials are used as plain text.

Optional features

Key	Description
L2Discovery	true enables the extra IP address custom properties (<code>L2interface</code> , <code>L2intId</code> , <code>L2VLAN</code> , <code>L2device</code>) used to track switch port/VLAN detail. Requires a matching <code>"deviceType": "L2"</code> entry in <code>profiles.json</code> plus BRIDGE-MIB support on the switch.
alertOnDeviceNotSeenInterval	Hours since last successful poll before a device is flagged. Also adds a monitor (Yes/No) custom property to devices, which can be toggled off per-device in the GUI.
alertOnDeviceNotSeenSyslog	<code>"IP FQDN:udp tcp:port"</code> — also send the not-seen alert to a syslog server, e.g. <code>"192.168.1.64:udp:514"</code> .
paloNAT (PaloAlto only)	true runs the PAN-OS <code>"show running nat-policy"</code> command over the API.
pNATProperty (PaloAlto only)	The IP address-level custom property used to store the destination NAT address, e.g. <code>"externalNAT"</code> . Populated as <code>timestamp * router name * NAT policy name</code> .
NSXTDiscovery	true enables the custom property definitions named in <code>NSXTSegmentProperties</code> and <code>NSXTPortProperties</code> below (or their defaults, if those aren't set) — see Appendix 6.7. Requires a matching <code>"discoveryType": "NSX-T"</code> entry in <code>profiles.json</code> .
NSXTSegmentProperties	Maps Micetro Range custom property names to NSX-T segment JSON fields, e.g. <code>{"nsxSegmentId": "id", "nsxReplicationMode": "replication_mode"}</code> . Dotted paths reach nested fields (e.g. <code>"advanced_config.multicast"</code>). Not sure which fields to capture yet? Leave unset: defaults to <code>nsxSegmentId</code> , <code>nsxTransportZonePath</code> , <code>nsxConnectivityPath</code> , and <code>nsxAdminState</code> . See Appendix 6.7 for what a raw segment looks like.
NSXTPortProperties	Same idea, for IP address custom properties sourced from the NSX-T port JSON. Defaults to <code>{"nsxPortId": "id"}</code> if unset.

4. Configuration — profiles.json

profiles.json describes the routers and switches to poll: an array of profile objects, each with its own discoveryType and credentials. It carries more flexibility than setup.json, so it takes a little more care to configure. That flexibility is what lets discovery be tailored to the environment, not just the standard SNMP MIBs.

Common fields (all profile types)

Field	Description
enabled	true / false — disabled profiles are skipped entirely.
profileName	A unique label used only for logging.
discoveryType	SNMP, NX-OS, PAN-OS, FORTI-OS, or NSX-T.
deviceType	Omit for L3 devices; set to "L2" to enable BRIDGE-MIB port discovery.
routers	SNMP/NX-OS/NSX-T: a list of IP addresses or FQDNs. PAN-OS/FORTI-OS: a list of URLs including port and credential (see below).

SNMP-specific fields

Field	Description
version	"2c" or "3".
communityString	Read-only community string (v2c only).
user, authProtocol, authPass, encrProtocol, encrKey	SNMPv3 credentials. Protocols map directly to pysnmp's UsmUserData class — see the pysnmp v3 docs at docs.lectudio.com for the full list of supported auth/privacy protocols.
oids	Overrides individual default OIDs for this profile (see 4.3).

API-specific fields (NX-OS / NSX-T / PAN-OS / FORTI-OS)

Field	Description
apiUser / apiPassword	NX-OS and NSX-T API credentials. NSX-T uses these to log in via its session API (see Appendix 6.7) rather than embedding a key in the URL.
routers (PAN-OS / FORTI-OS)	Each entry is "https://host:port:apiKey", e.g. https://10.50.0.214:443:<key>.

4.1 Default OIDs used by Micetro

Name	MIB	OID
ipNetToMediaPhysAddress	IP-MIB	.1.3.6.1.2.1.4.22.1.2
ipAdEntAddr	IP-MIB	.1.3.6.1.2.1.4.20.1.1
ipAdEntIfIndex	IP-MIB	.1.3.6.1.2.1.4.20.1.2
ipAdEntNetMask	IP-MIB	.1.3.6.1.2.1.4.20.1.3
ifDescr	IF-MIB	.1.3.6.1.2.1.2.2.1.2

Name	MIB	OID
ifAlias	IF-MIB	.1.3.6.1.2.1.31.1.1.1.18
entPhysicalSerialNum	ENTITY-MIB	.1.3.6.1.2.1.47.1.1.1.1.11
sysName	SNMPv2-MIB	.1.3.6.1.2.1.1.5
cvroutedVlanIfIndex	CISCO-VLAN-IFTABLE-RELATIONSHIP-MIB	.1.3.6.1.4.1.9.9.128.1.1.1.1.3
cvVrfInterfaceType	CISCO-VRF-MIB	.1.3.6.1.4.1.9.9.711.1.2.1.1.2
cvVrfName	CISCO-VRF-MIB	.1.3.6.1.4.1.9.9.711.1.1.1.1.2

Note: Any of these can be overridden per-profile via the oids object — useful when a vendor implements a MIB slightly differently (see the Cisco Firepower example below).

4.2 Worked examples

Basic SNMPv2c against one switch

```
{
  "profiles": [
    {
      "enabled": true,
      "profileName": "mySwitch",
      "discoveryType": "SNMP",
      "version": "2c",
      "communityString": "public",
      "routers": ["192.168.1.1"]
    }
  ]
}
```

Same, with additional routers – works for all profile types!

```
"routers": ["192.168.1.1", "192.168.1.10", "192.168.10.1"]
```

Disabled profile (skipped at runtime)

```
{
  "enabled": false,
  "profileName": "mySwitch",
  "discoveryType": "SNMP",
  "version": "2c",
  "communityString": "public",
  "routers": ["192.168.1.1"]
}
```

Overriding a default OID (Cisco Firepower)

Firepower exposes ARP data under a different OID than Micetro's default. Overriding just that one key is enough:

```
{
  "enabled": true,
  "profileName": "ciscoFirePower",
  "discoveryType": "SNMP",
  "version": "2c",
  "communityString": "public",
  "oids": { "ipNetToMediaPhysAddress": "1.3.6.1.2.1.4.35.1.4" },
}
```



```
{
  "routers": ["192.168.45.147"]
}
```

PAN-OS — two firewalls via API key

```
{
  "enabled": true,
  "profileName": "PaloAlto",
  "discoveryType": "PAN-OS",
  "routers": [
    "https://10.50.0.120:443:<api_key_1>",
    "https://10.50.0.100:443:<api_key_2>"
  ]
}
```

Cisco Nexus — API credentials

```
{
  "enabled": true,
  "profileName": "Nexus",
  "discoveryType": "NX-OS",
  "apiUser": "micetro",
  "apiPassword": "micetro123",
  "routers": ["10.10.10.6"]
}
```

NSX-T — Manager credentials

```
{
  "enabled": true,
  "profileName": "nsxLab",
  "discoveryType": "NSX-T",
  "apiUser": "nsxmgr ",
  "apiPassword": "password123",
  "routers": ["nsx-mgr.example.com"]
}
```

SNMPv3 (MD5 / DES) — credentials already encrypted

After the first run with `encryptAuth: true`, plain-text credentials are replaced automatically with Fernet-encrypted values (the `EEEEEE#` prefix marks them as encrypted):

```
{
  "enabled": true,
  "profileName": "netgear3",
  "discoveryType": "SNMP",
  "version": "3",
  "user": "v3user",
  "authProtocol": "usmHMACMD5AuthProtocol",
  "authPass": "EEEEEE#gAAAAABmoi-Lb8sj...",
  "encrProtocol": "usmDESPrivProtocol",
  "encrKey": "EEEEEE#gAAAAABmoi-Lo0Kp...",
  "routers": ["10.50.0.5"]
}
```

Layer-2 discovery via BRIDGE-MIB

Setting `deviceType` to "L2" tells the script to also walk the BRIDGE-MIB and populate the L2VLAN, L2device, and L2interface custom properties where possible:

```
{
  "enabled": true,
```

```
"profileName": "bridge",  
"discoveryType": "SNMP",  
"deviceType": "L2",  
"version": "2c",  
"communityString": "routers",  
"routers": ["10.50.0.8"]  
}
```

Note: L2 port mapping relies on MAC/IP data already learned from upstream L3 devices. If more than one MAC is seen on a switch port, that port is assumed to lead to a downstream device and is skipped. List upstream L3 devices before L2 devices in profiles.json so their ARP data is available first.

4.3 Validating the file

Before deploying, confirm the JSON is well-formed:

```
python -m json.tool profiles.json
```

Editors such as Visual Studio Code will also flag syntax errors as you type.

5. Installation

5.1 Copy files to the scripts folder

Copy externalDiscovery (or .exe), profiles.json, and setup.json into the Micetro Central scripts folder:

- Windows: C:\ProgramData\Men and Mice\Central\scripts
- Linux: /var/mmsuite/mmcentral/scripts

Note: Finish tailoring and validating both JSON files before copying them across.

5.2 Disable the built-in discovery

Running both the built-in engine and externalDiscovery against the same devices will cause conflicting updates. Under ADMIN → CONFIGURATION → SNMP PROFILES, confirm there are no active profiles, or that none have routers assigned.

5.3 Prepare Micetro

- Create a ScriptRunner account with the administrator role. Micetro generates a unique password for it on each script run for security — you don't manage this password directly.
- Under System Settings, confirm "Always show discovery info" is enabled.

5.4 Schedule the script

- Go to ADMIN → CONFIGURATION → EVENT HOOKS → Scheduled Events → + ADD
- The compiled binary should appear in the dropdown once it's in the scripts folder
- Enable it, tick "Add ScriptRunner authentication parameters" (required for API writes), and set a recurring schedule — hourly is typical

Note: The script's own PID-file check means a shorter interval is safe: if a previous run is still in progress, the new run exits gracefully instead of overlapping.

5.5 Test run

- Select "Run now" → confirm "Yes"
- It should complete without error

5.6 Check the log

Log verbosity depends on logLevel in setup.json. The default log location is one directory above Central, though both name and path are configurable. Here's a DEBUG-level excerpt against a Netgear switch:

```
2024-07-25 11:57:19,400 - Processing:10.50.0.5 in profile netgear2
2024-07-25 11:57:19,400 - Using SNMP version: 2c
2024-07-25 11:57:19,588 - Processing interfaces
2024-07-25 11:57:20,072 - Processing interface descriptions
2024-07-25 11:57:20,353 - Getting Network data from 10.50.0.5
2024-07-25 11:57:20,697 - No current networks in Micetro matching router: M4100S
2024-07-25 11:57:20,713 - Updating subnet 10.40.0.0/24
2024-07-25 11:57:20,775 - Network Updates Complete
2024-07-25 11:57:20,838 - Getting ARP data from 10.50.0.5
2024-07-25 11:57:21,056 - Discovered IP/MAC: 10.40.0.1/60:38:e0:d8:c4:f3
2024-07-25 11:57:21,088 - Processing Router Profile: vyos
```

6. Appendix

6.1 Troubleshooting

Decryption failures

If the log shows repeated Fernet decryption failures, the credentials in profiles.json were encrypted with a different Fernet key than the one currently stored in the fernetUser account's description field. That typically happens after the field is cleared or the account rebuilt.

```
2024-07-25 09:46:01,038 - authpass encrypted. unpacking using FernetKey
2024-07-25 09:46:01,038 - Failed to decrypt credential using Fernet. Was this encrypted using a
different Fernet key?
```

Fix: delete the Fernet key from the description field and reset the affected credentials in profiles.json back to plain text. The next run will generate a new key and re-encrypt everything.

Backing up the database before testing

```
# Linux
systemctl stop mmcentral
cd /var/mmsuite/mmcentral
cp mmsuite.db mmsuite.bak
systemctl start mmcentral
```

6.2 Testing OIDs with snmpwalk

Useful for finding a vendor-specific replacement for a default OID. Cisco Firepower, for example, returns comparable ARP data from 1.3.6.1.2.1.4.35.1.4 instead of the Micetro default 1.3.6.1.2.1.4.22.1.2.

```
snmpwalk -v 2c -c <community_string> <router_ip> <oid>
```

SNMPv3 example:

```
snmpwalk -v 3 -l authPriv -a SHA -u v3user -A "v3userpass" -x AES -X "12345678" 192.168.12.3
1.3.6.1.2.1.1.5
SNMPv2-MIB::sysName.0 = STRING: vrcore0
```

SNMPv2c ARP query:

```
snmpwalk -v 2c -c routers 10.50.0.4 1.3.6.1.2.1.4.22.1.2
IP-MIB::ipNetToMediaPhysAddress.66.10.40.0.1 = STRING: 60:38:e0:d8:c4:f3
IP-MIB::ipNetToMediaPhysAddress.66.10.40.0.3 = STRING: 0:30:18:8:2:c5
```

Full default OID reference: docs.menandmice.com/en/latest/guides/reference/snmp_oid/

6.3 FORTI-OS notes

The FortiOS API has no direct ARP table endpoint. As a workaround, the script combines two endpoints to build an equivalent IP/MAC picture:

Endpoint	Purpose
/api/v2/monitor/system/dhcp	Leased DHCP addresses with MAC, hostname, and interface.
/api/v2/monitor/system/interface	Interface IPs and MACs, including addresses not covered by DHCP.

Example DHCP response (trimmed):

```
{
  "results": [
    {
      "ip": "10.100.0.21",
      "mac": "00:0c:29:99:9a:62",
      "hostname": "box",
      "interface": "port2",
      "status": "leased"
    }
  ],
  "status": "success"
}
```

Example interface response (trimmed):

```
{
  "results": {
    "port1": {
      "mac": "00:0c:29:bc:99:1a",
      "ip": "10.50.0.250",
      "mask": 24
    }
  },
  "status": "success"
}
```

6.4 Building standalone binaries (PyInstaller)

PyInstaller (pyinstaller.org) bundles the script with its dependencies so the target machine needs no Python install and the source stays hidden. Build on a matching OS with all dependencies installed first.

- Extra packages required beyond a base Python install: requests, cryptography (json, ipaddress, and logging ship with the standard library)

```
pyinstaller -F externalDiscovery.py
```

The -F flag produces a single-file executable.

6.5 API commands reference

Cisco Nexus (NX-API)

Task	CLI command sent via API
Get hostname	show hostname
Get interfaces (incl. VRF)	show ip interface vrf all
Get ARP entries (incl. VRF)	show ip arp detail vrf all

Reference: [developer.cisco.com](https://developer.cisco.com/docs/nexus-api/) — NX-API CLI reference for Nexus 7000

Palo Alto (PAN-OS XML API)

Task	Call
Get hostname	/api/?type=op&cmd=<show><system><info></info></system></show>
Get interfaces	/api/?type=op&cmd=<show><interface>all</interface></show>
Get management interface	/api/?type=op&cmd=<show><interface>management</interface></show>
Get ARP entries	/api/?type=op&cmd=<show><arp><entry name='all'/></arp></show>
Get NAT rules	/api/?type=config&action=get&xpath=/config/devices/entry[@name='localhost.localdomain']/vsys/entry[@name='vsys1']/rulebase/nat
Generate an API key	/api/?type=keygen&user=<username>&password=<password>

Reference: docs.paloaltonetworks.com — PAN-OS REST API getting started guide

6.6 Simulating a device for testing

When a real device of the target type isn't available, a local HTTP server can stand in for it using sample output supplied by the customer:

- Create a subdirectory named api
- Save the sample XML/JSON response as api/index.html
- Start a local web server:

```
python3.10 -m http.server -b 127.0.0.1 8000
```

- Point the script at localhost:8000 for testing

Note: This only simulates the response payload, not authentication — useful for confirming parsing logic, not end-to-end connectivity.

6.7 NSX-T notes

NSX-T doesn't keep this in one place. Networks come from one API call, but realized IP/MAC bindings for active VMs need a call per port with no bulk option -- two other endpoints looked like the right answer and weren't (see the note below the table):

Endpoint	Purpose
GET /policy/api/v1/infra/segments	Segments and their subnets — becomes the network/Range data, tagged with nsxSegmentId, nsxTransportZonePath, nsxConnectivityPath, and nsxAdminState custom properties.

Endpoint	Purpose
GET /policy/api/v1/infra/segments/{segment-id}/ports	Lists the ports on a segment: one per VM's virtual NIC (VIF), plus router/uplink ports.
GET /policy/api/v1/infra/segments/{segment-id}/ports/{port-id}/state	Discovered and realized IP/MAC bindings for that one port (the ARP-equivalent), each tagged with a source (ARP_SNOOPING, VM_TOOLS, ...). One call per port, no bulk option. Populates the nsxPortId custom property on each discovered IP address.

Note: Two other endpoints looked right and weren't. The Policy API's own SegmentPort.attachment.address_bindings only ever carries explicit static bindings: it's empty for ordinary VM ports. The Manager API's per-switch mac-table (/policy/api/v1/infra/realized-state/{logical-switch-id}/mac-table) does return real data, but it's MAC-to-VTEP (which ESXi host the MAC was learned behind), not the VM's own IP. The port state endpoint above is the one that actually has both.

Authentication is session-based rather than a static API key or Basic auth: the script logs in via POST /api/session/create with apiUser/apiPassword, and uses the returned X-XSRF-TOKEN header (plus session cookie) on every subsequent call.

skipNetworks (see section 3) works here too. It's useful for excluding NSX's auto-plumbed Tier-1 router-link segments: small subnets (e.g. a /28) often shared identically across many unrelated segments, not real workload networks.

Which NSX-T metadata actually lands as a custom property is a config choice, not a code change.

NSXTSegmentProperties and NSXTPortProperties in setup.json (see section 3) map Micetro property names to fields in the raw segment/port JSON, including nested ones via a dotted path:

```
{
  "NSXTSegmentProperties": {
    "nsxSegmentId": "id",
    "nsxReplicationMode": "replication_mode",
    "nsxMulticast": "advanced_config.multicast"
  },
  "NSXTPortProperties": {
    "nsxPortId": "id",
    "nsxAttachmentType": "attachment.attachment_type"
  }
}
```

Note: Leave either key out of setup.json and it falls back to the original defaults: nsxSegmentId, nsxTransportZonePath, nsxConnectivityPath, and nsxAdminState for segments; nsxPortId for ports. Existing deployments don't need to change anything.